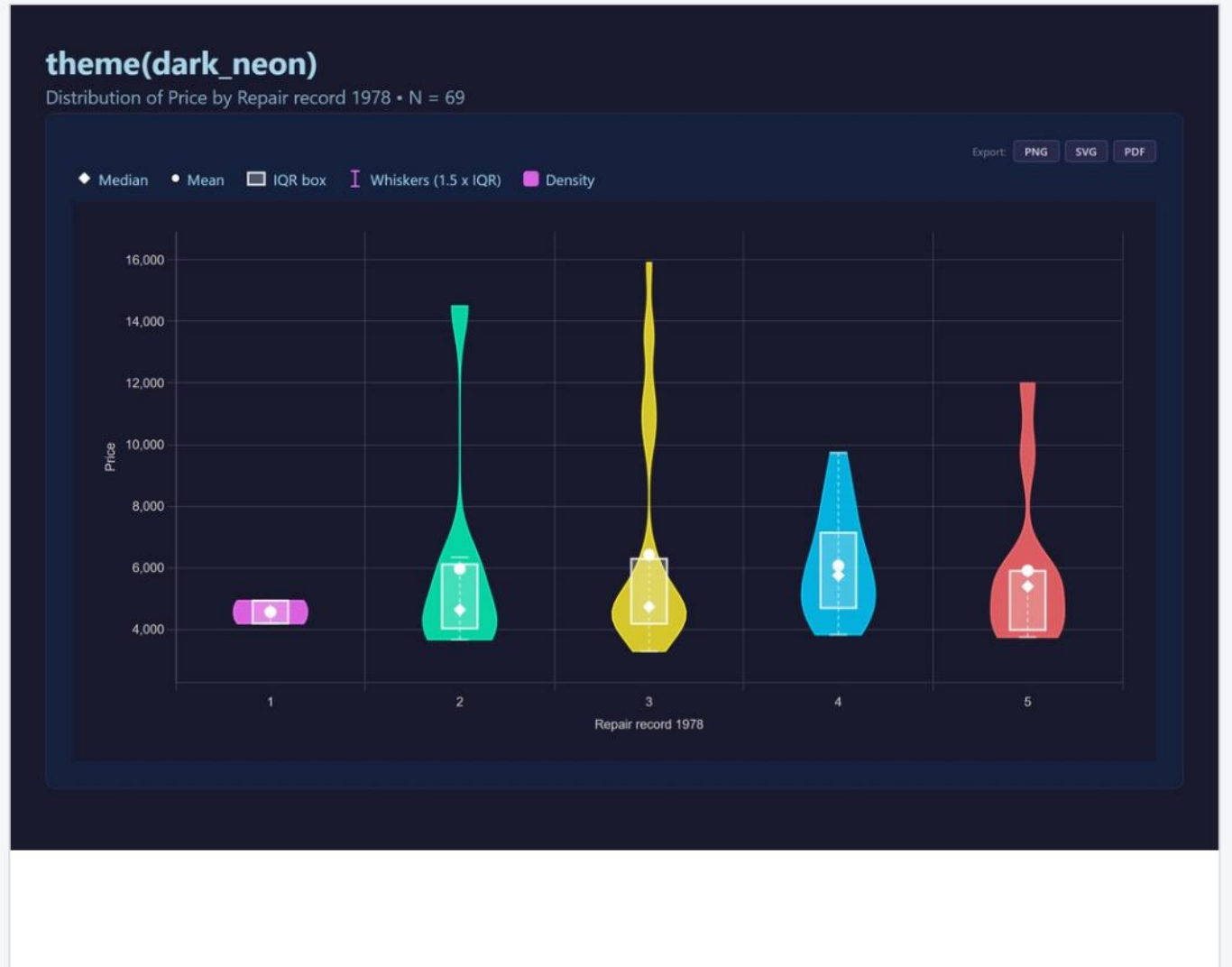


sparkta

An interactive graph command for Stata

Fahad Mirza, World Bank
Stata Conference, October 2026
sparkta 3.6.0



Same command, same data. One of them you can touch.

Stata graph, a picture



```
. graph bar price, over(rep78)
```

sparkta, a page you can use



```
. sparkta price, over(rep78)
```

Why I built sparkta

One program, not three. On locked-down institutional systems such as the World Bank's, it is rare to find a user with Stata, R and Python all installed and maintained. Stata is the one people have.

Sensitive data stays on the machine. Everything is computed in Stata and written to one file locally: no upload, no cloud service, no server.

“Can Stata do interactive charts?” Users kept asking. In R and Python, plotly, htmlwidgets and Bokeh all work the same way: the host language writes a JavaScript chart into a web page. sparkta does that from Stata, through the Java bridge Stata already ships.

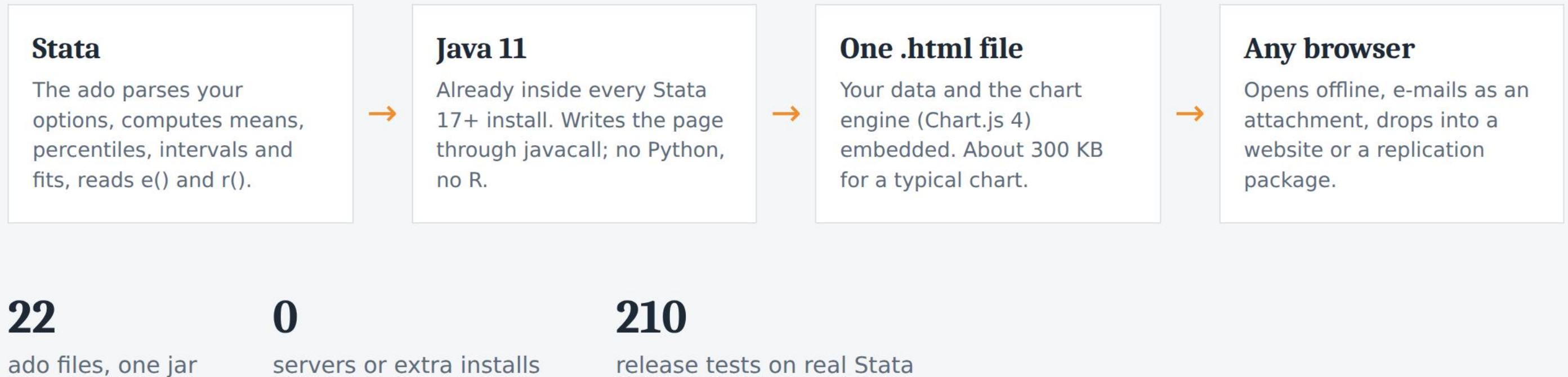
One workflow, easy to audit. Data, estimates and charts come from the same do-file and the same log; rerun it and you get the same page.

If you know graph, you already know sparkta

Stata word	in sparkta
<code>title()</code> <code>subtitle()</code> <code>note()</code> <code>xtitle()</code> <code>ytitle()</code>	same word
<code>over()</code> <code>by()</code> <code>legend()</code> <code>yline()</code> <code>xline()</code> <code>horizontal</code>	same word
<code>level(95)</code> <code>keep()</code> <code>drop()</code> <code>msymbol()</code> <code>mcolor()</code> <code>lwidth()</code>	accepted as aliases
<code>mcolor(navy maroon)</code> , <code>colorpalette</code> names	Stata colour names, hex, %opacity, *intensity
<code>scheme()</code>	<code>theme()</code> light, dark, colour-blind safe
<code>saving()</code> / graph export	<code>export(file.html)</code> and <code>saveas(file.png pdf svg)</code>

Full mapping: docs/OPTIONS_VS_STATA.md in the repository.

What happens when you press Enter



Why Java, and not Python or R?

	Java (sparkta)	Python or R route
Install	already inside every Stata 17+; nothing to set up	install Python or R, add packages, point Stata at them
Portability	one jar of about 400 KB; the same file on every platform Stata runs on	an environment per machine; package versions drift
Data	reads variables in place through Stata's Java API	Python's sfi can too; R needs an export or a bridge
Speed	compiled; a 100,000-point page in about 6 seconds	fast, once the environment exists
Output	one offline HTML file	often a server (Shiny, Dash) or online libraries

Python and R are excellent; the point is zero friction for a Stata user, inside a do-file.

100% offline, by design

What institutions need	What sparkta does
No internet, no server	Chart engine and data are embedded in one HTML file; it opens from a USB stick, a shared drive or an e-mail. Online mode is opt-in only.
Locked-down machines	Nothing to install beyond the Stata package; no admin rights, no Python or R, no cloud account.
Data stays in-house	Nothing is uploaded anywhere: the page is built on your machine and only travels where you send it.
Reproducible and archivable	The same do-file rebuilds the same page; it drops into replication packages and still opens years later.
Free and open	MIT licence: use it, change it, ship it in your own tools.

One caution: the page carries the rows its filters need, so share pages of confidential microdata only as you would the data, or aggregate first.

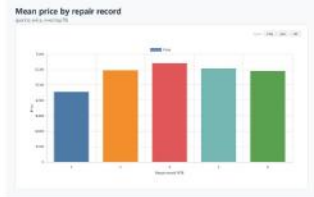
Part 1 of 5

Chart types

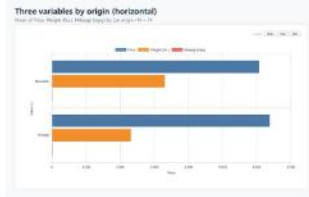
The graph bar, twoway, histogram and graph box family, plus a few Stata never had.

Eighteen chart types (and a step option) from one command

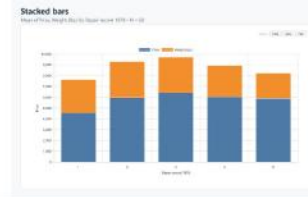
Click a tile to open the live page.



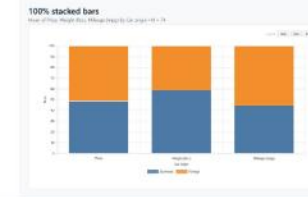
`type(bar)`



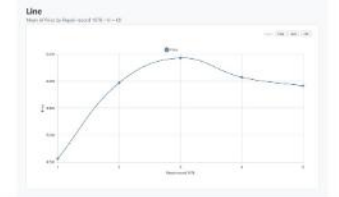
`type(hbar)`



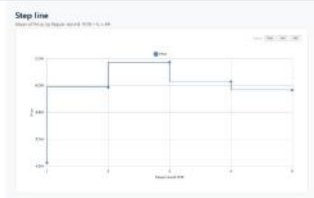
`type(stackedbar)`



`type(stackedbar100)`



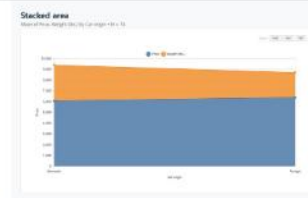
`type(line)`



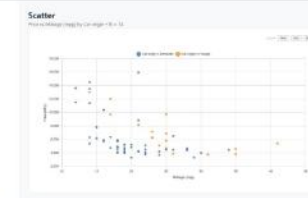
`type(line) stepped(after)`



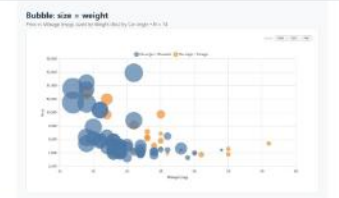
`type(area)`



`type(stackedarea)`



`type(scatter)`



`type(bubble)`



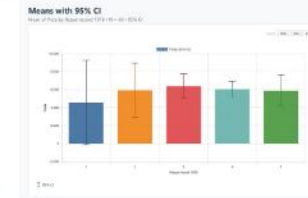
`type(pie)`



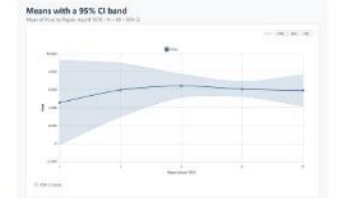
`type(donut)`



`type(histogram)`



`type(cibar)`



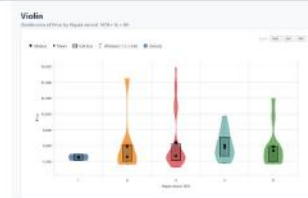
`type(ciline)`



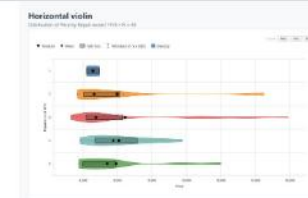
`type(boxplot)`



`type(hbox)`



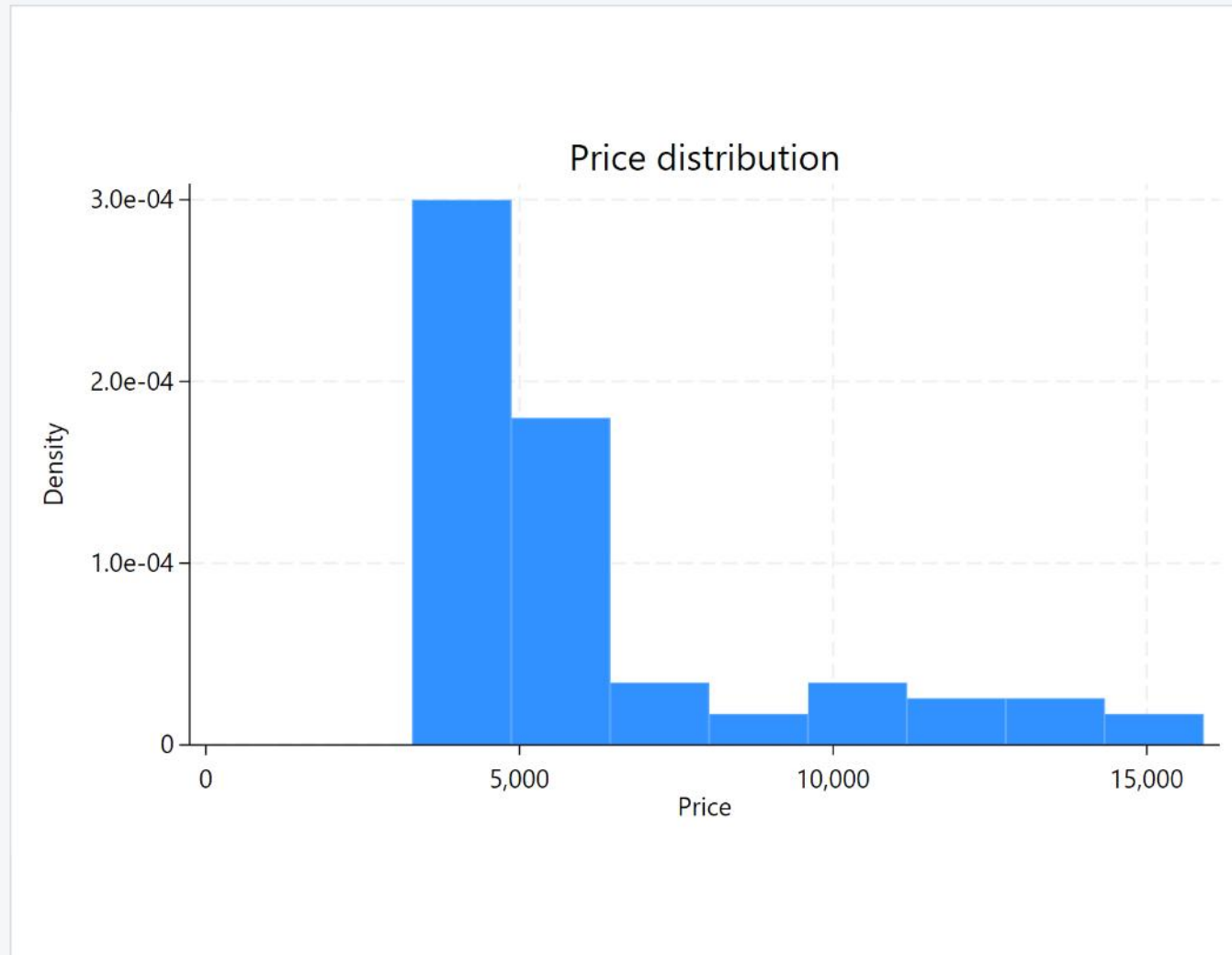
`type(violin)`



`type(hviolin)`

The numbers are Stata's numbers

Stata graph, a picture

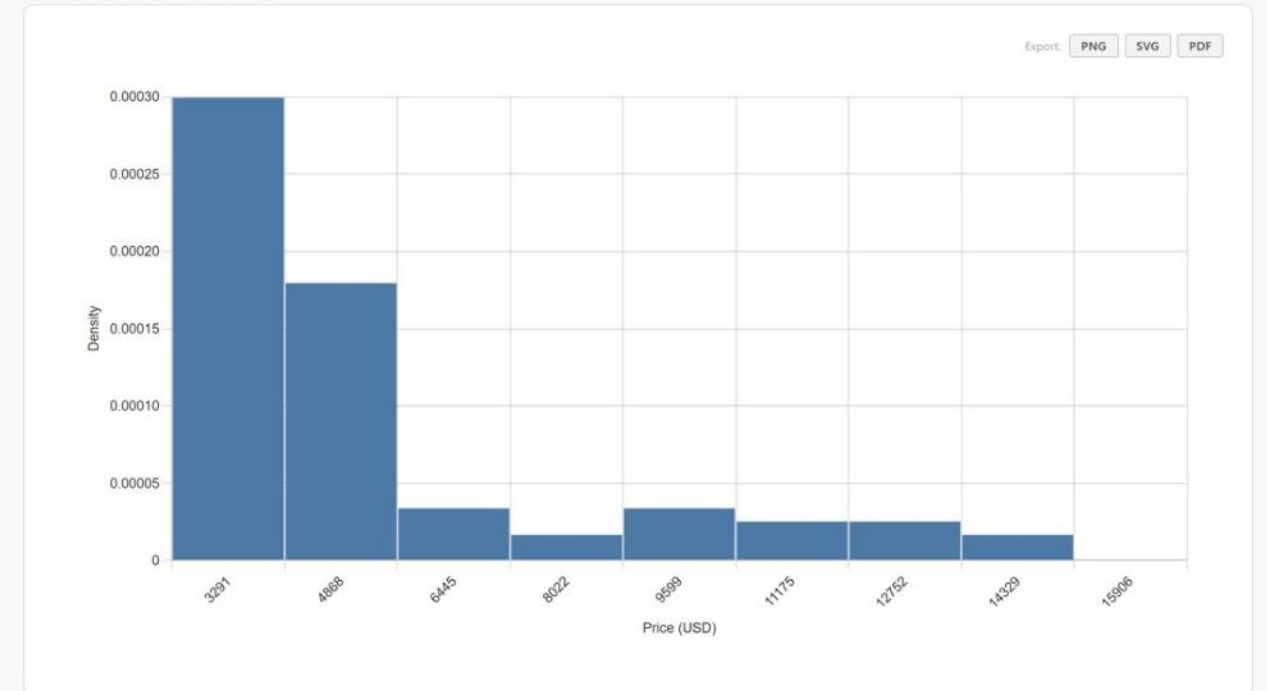


```
. histogram price
```

sparkta, a page you can use

Histogram

Distribution of Price • N = 74



```
. sparkta price, type(histogram)
```

Part 2 of 5

Interactivity

What no static graph can do: the reader asks the next question themselves.

Dropdowns that redraw every panel

`filters()` adds one dropdown per variable. With `by()`, every panel and its N redraw together, and each bar's tooltip gives its mean and n.

```
. sysuse nlsw88, clear  
. sparkta wage, over(race) by(collgrad) ///  
  filters(married union)
```

Pick married, then union members: both panels change. Hover a bar for its n.

Hourly wage by race, college graduates vs not

Two dropdowns redraw both panels; hover a bar for its n

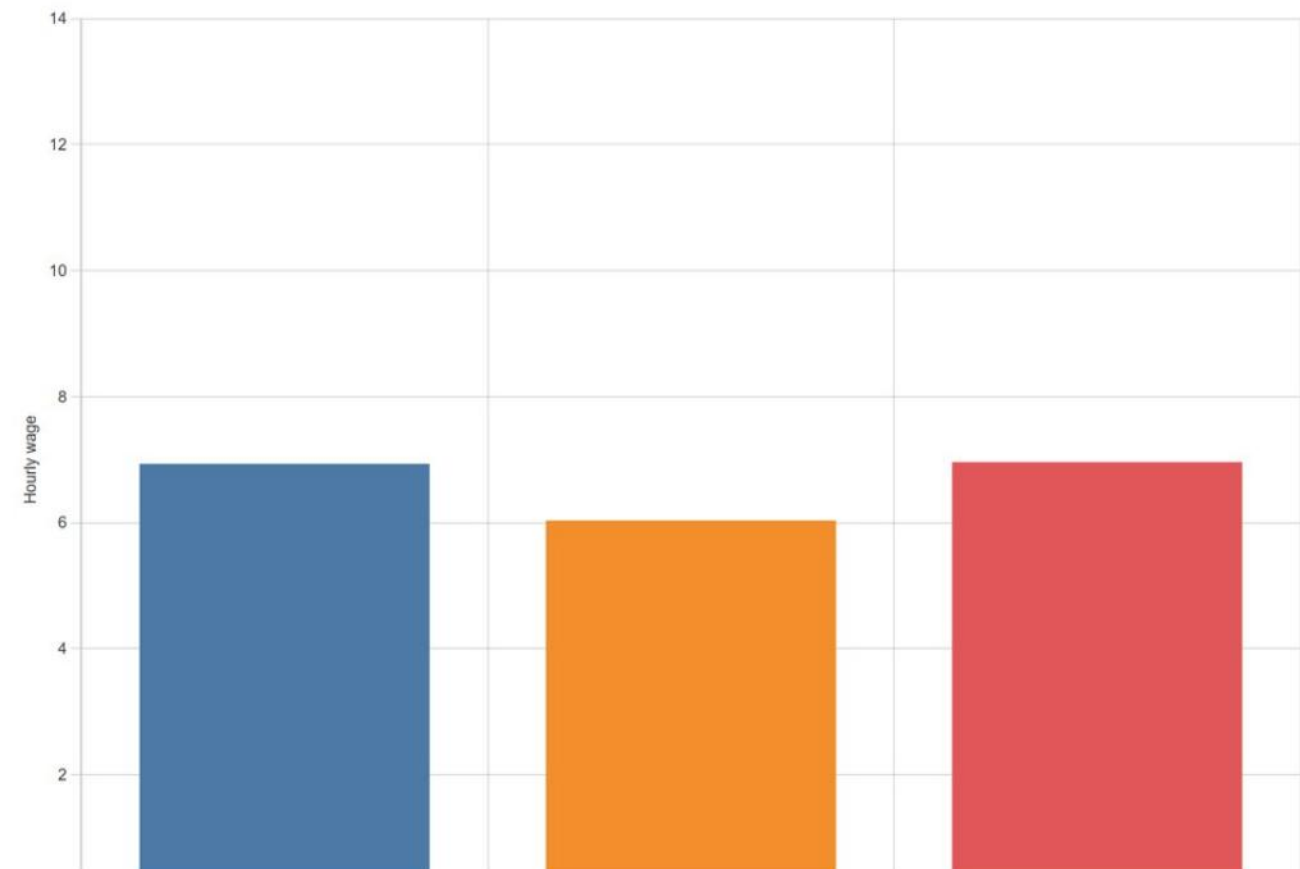
Married Union worker

Export:

■ White ■ Black ■ Other

College graduate = Not college grad

n = 1414



Sliders recompute the statistics, not just hide points

`sliders()` adds a two-handled range. Means and 95% intervals are recalculated from the rows that remain.

```
. sparkta price, type(cibar) over(rep78) ///  
  filters(foreign) sliders(mpg)
```

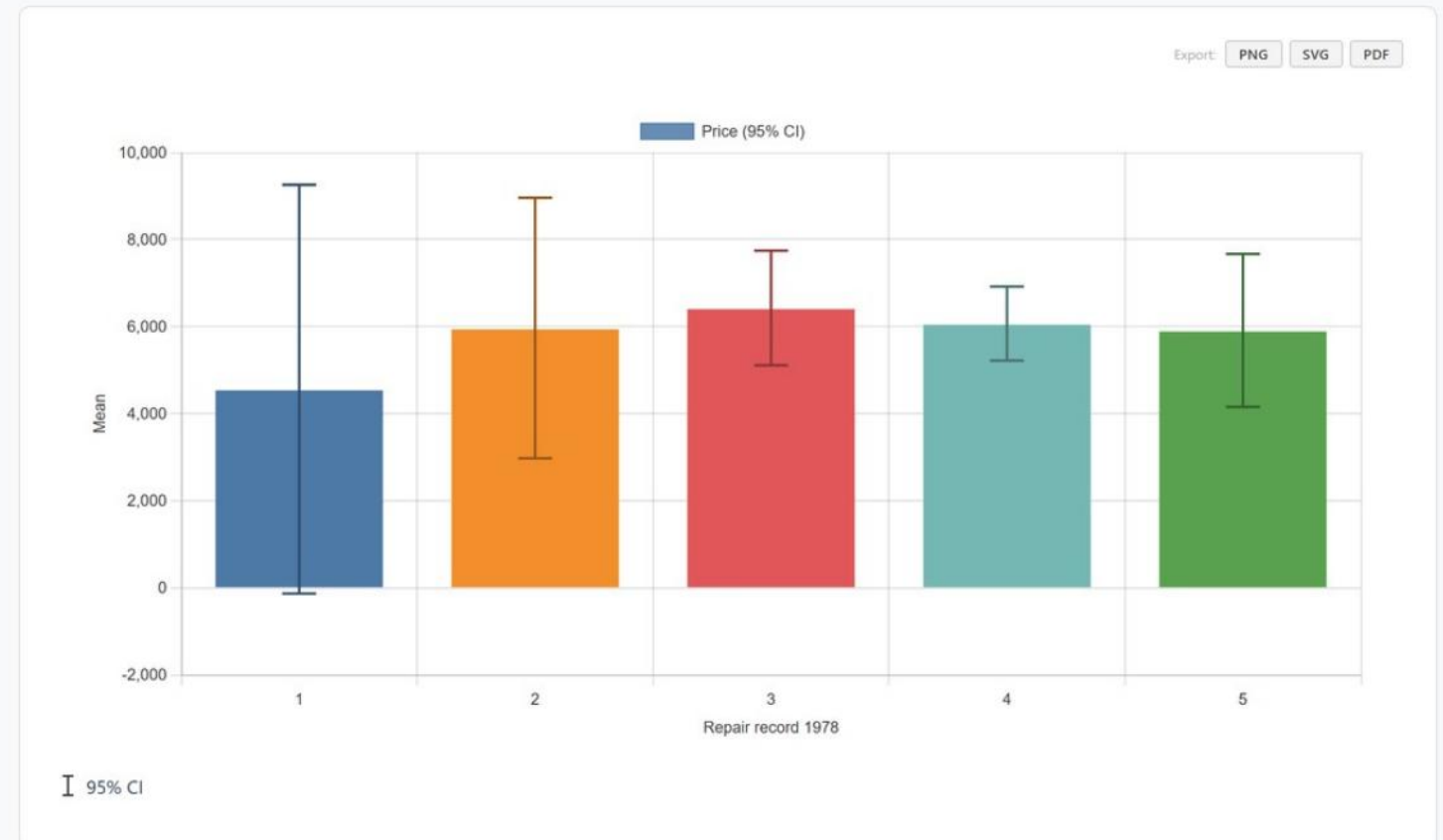
Drag the mpg handles inward: the intervals widen as N falls.

Means with 95% CI -- filter and slider

Intervals are recomputed for the rows you keep

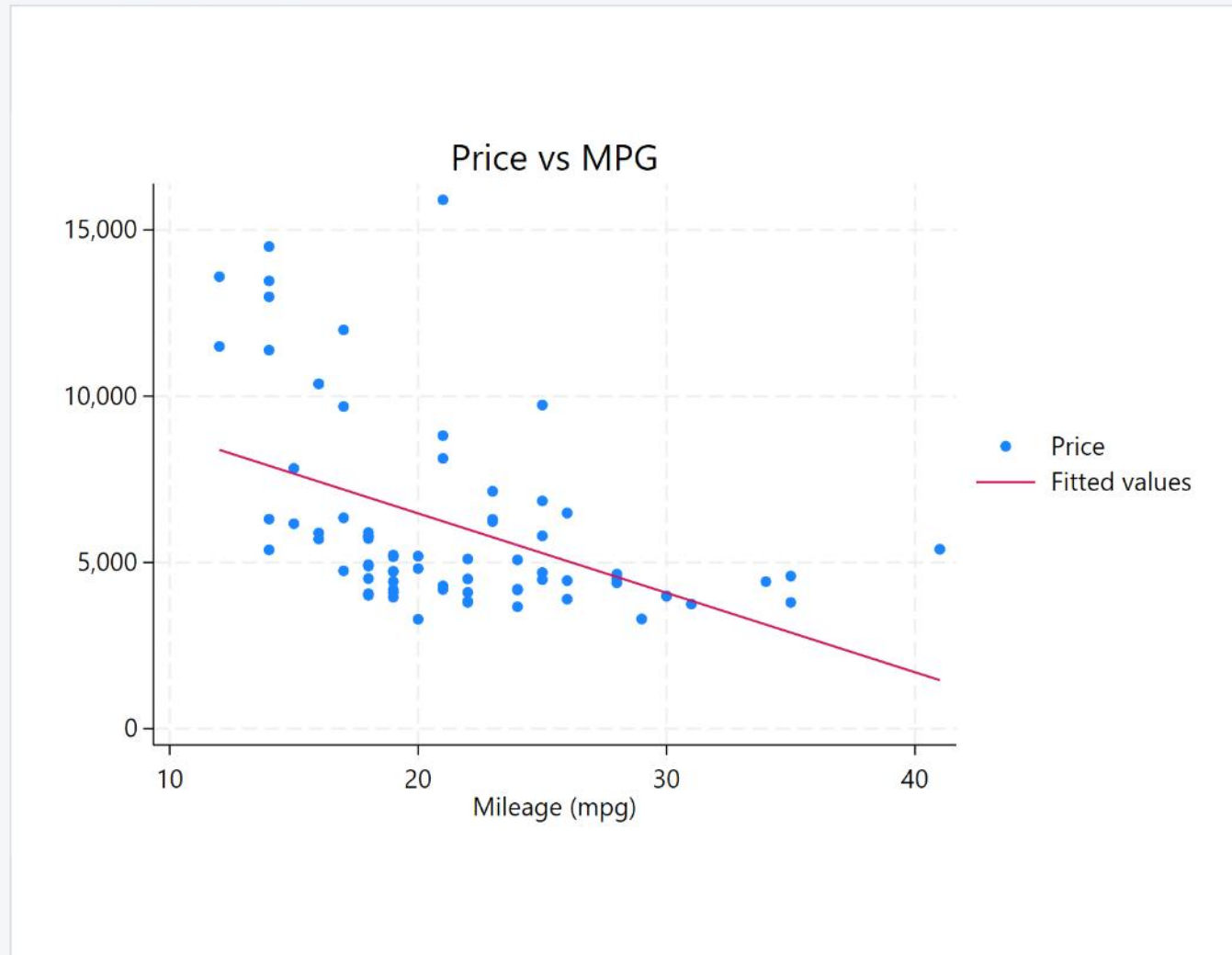
Car origin

Mileage (mpg)



The same fitted line as twoway lfit

Stata graph, a picture

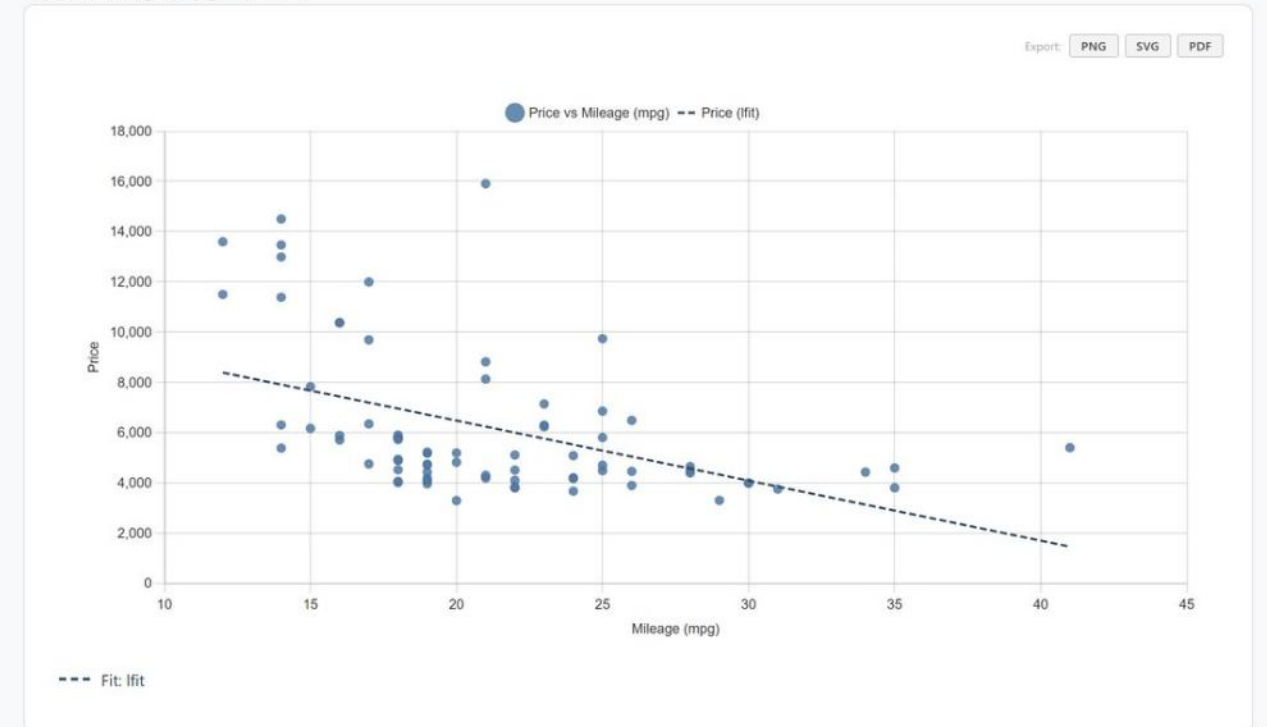


```
. twoway (scatter price mpg) (lfit price mpg)
```

sparkta, a page you can use

Price vs MPG

Price vs Mileage (mpg) • N = 74



```
. sparkta price mpg, type(scatter) fit(lfit)
```

Fit lines that follow the filter

The fit is re-estimated on the rows you keep: drag the weight slider and the LOWESS lines, points and N move together.

```
. sparkta price mpg, type(scatter) ///  
  over(foreign) fit(lowess) ///  
  filters(rep78) sliders(weight)
```

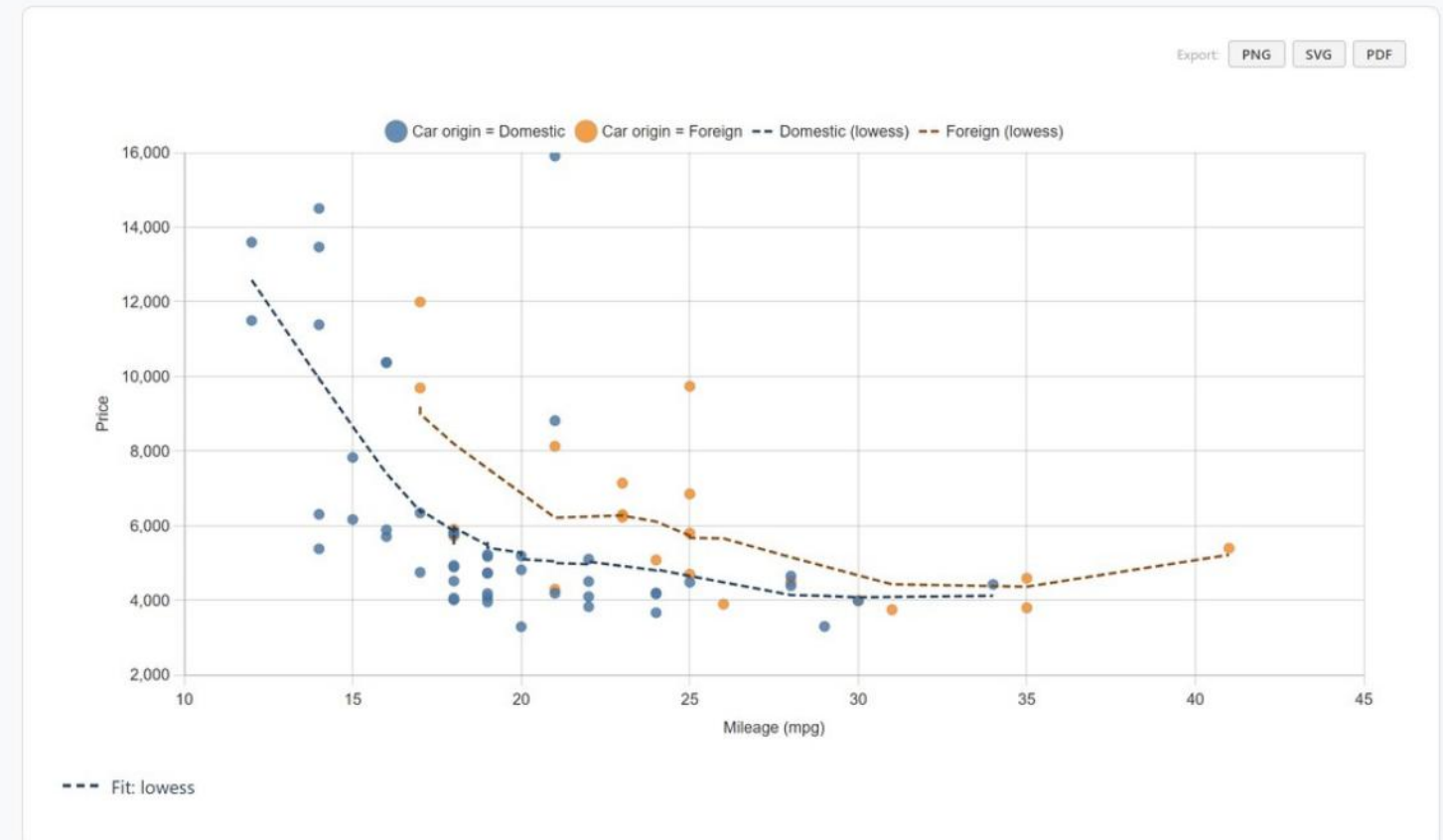
Drag the weight slider to heavy cars only and watch both smooths change.

Price vs MPG with a LOWESS smooth

Drag the weight slider: points, fit line and N update

Repair record 1978

Weight (lbs.)



On real survey data

nls88, 2,246 women. One slider on total experience, a fitted line per education group.

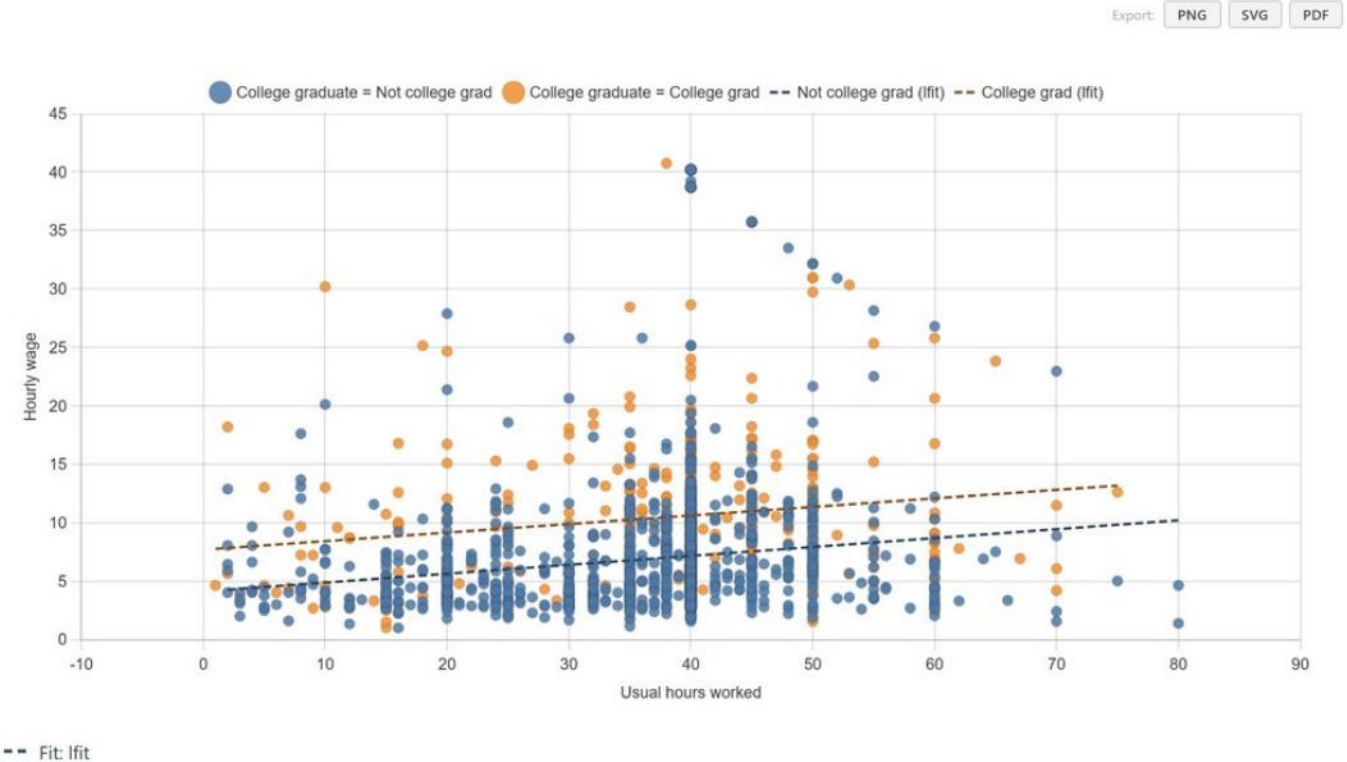
```
. sysuse nls88, clear  
. sparkta wage hours, type(scatter) over(collgrad) ///  
  fit(lfit) sliders(ttl_exp)
```

Wages vs hours by education

Drag the experience slider

Filters: Total work experience (years) = 0.12 - 28.88

Total work experience (years)  0.12 - 28.88



Slide experience to 15+ years and compare the two slopes.

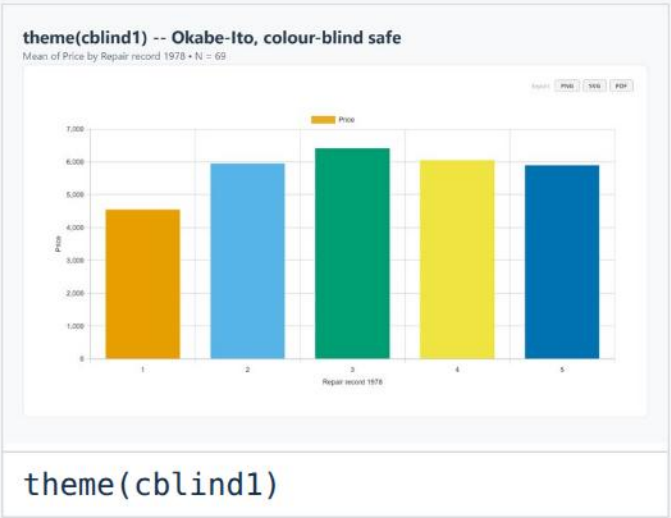
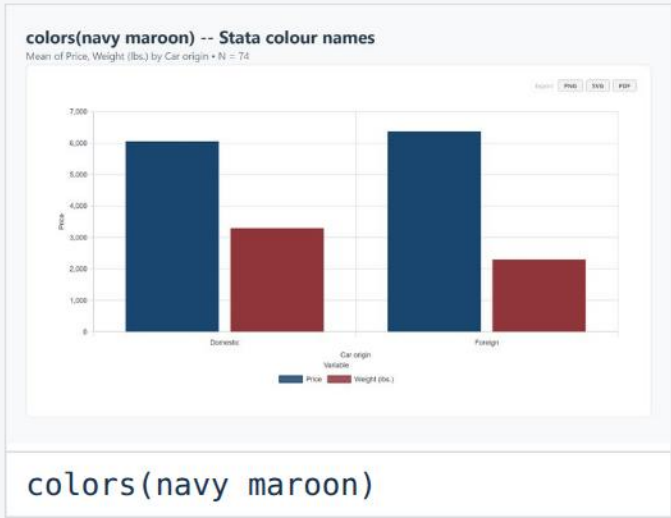
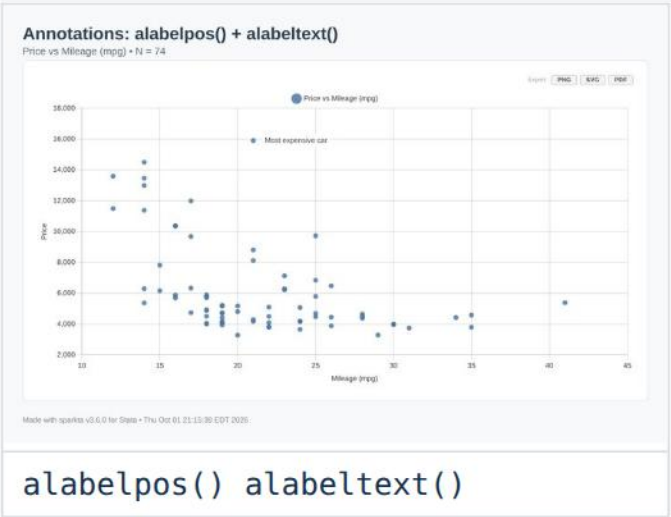
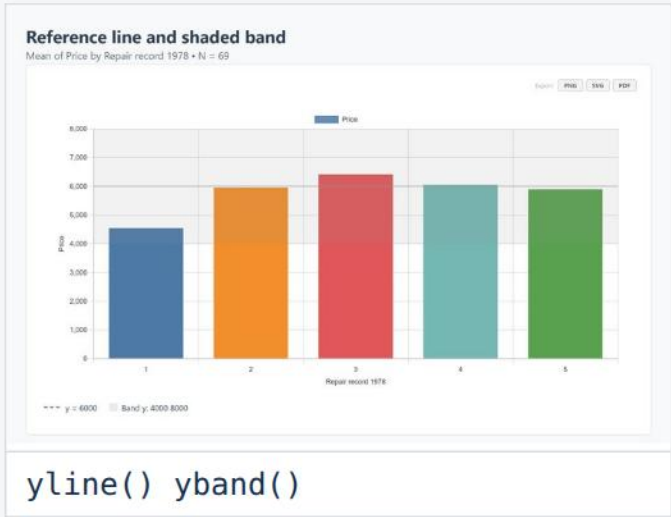
Part 3 of 5

Styling

The options you already type after the comma.

Reference lines, labels, annotations, colours, themes

Click a tile to open the live page.



Part 4 of 5

After the regression

coefplot, marginsplot and event studies, read straight from `e()` and `r()`.

Two models, one chart, one publication table

No varlist: the chart reads e(b) and e(V) of stored estimates (z_ = standardised, one SD). The table under it copies as Markdown or LaTeX and downloads as CSV, a standalone .tex file or PDF.

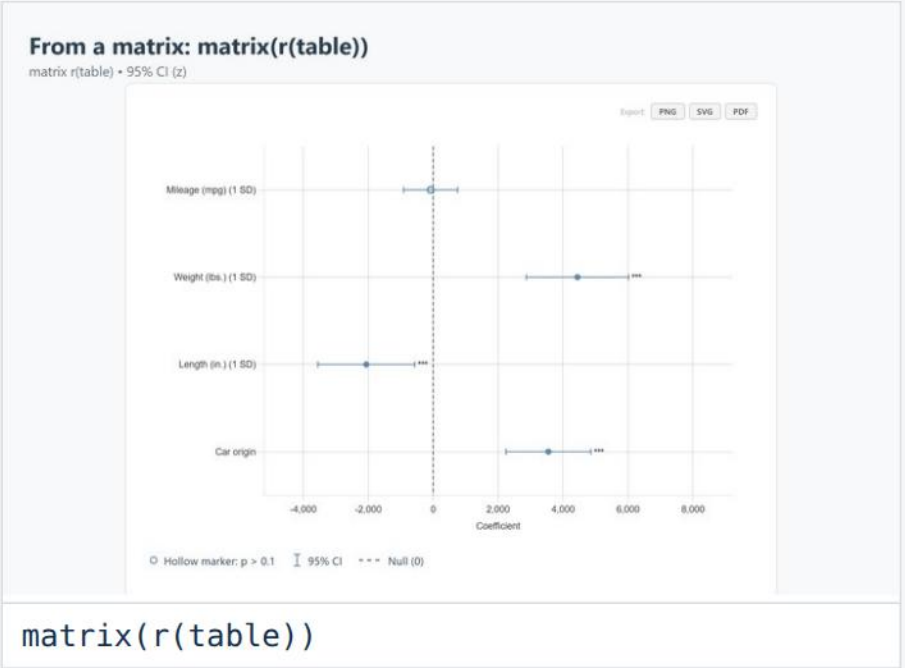
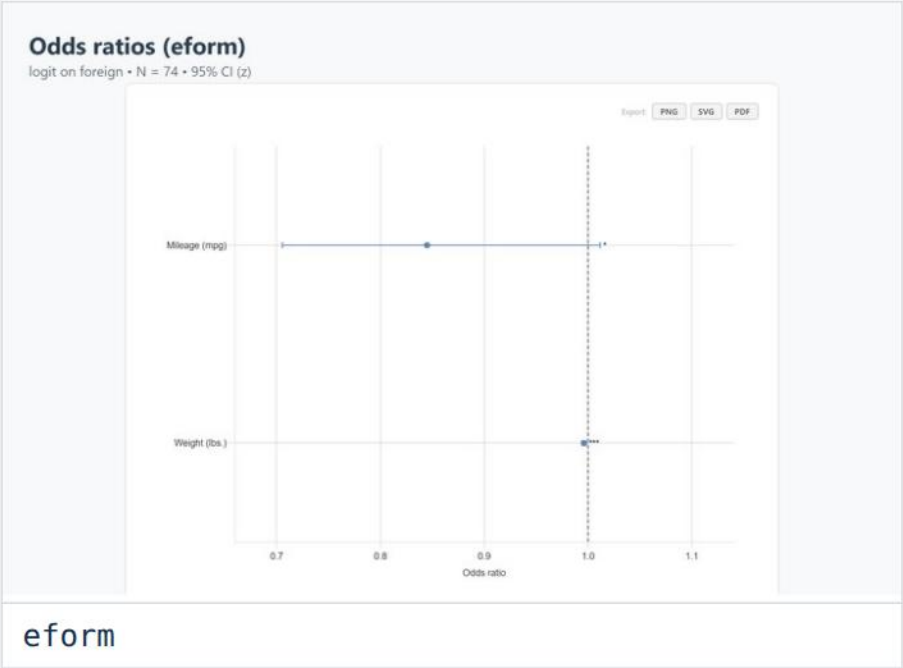
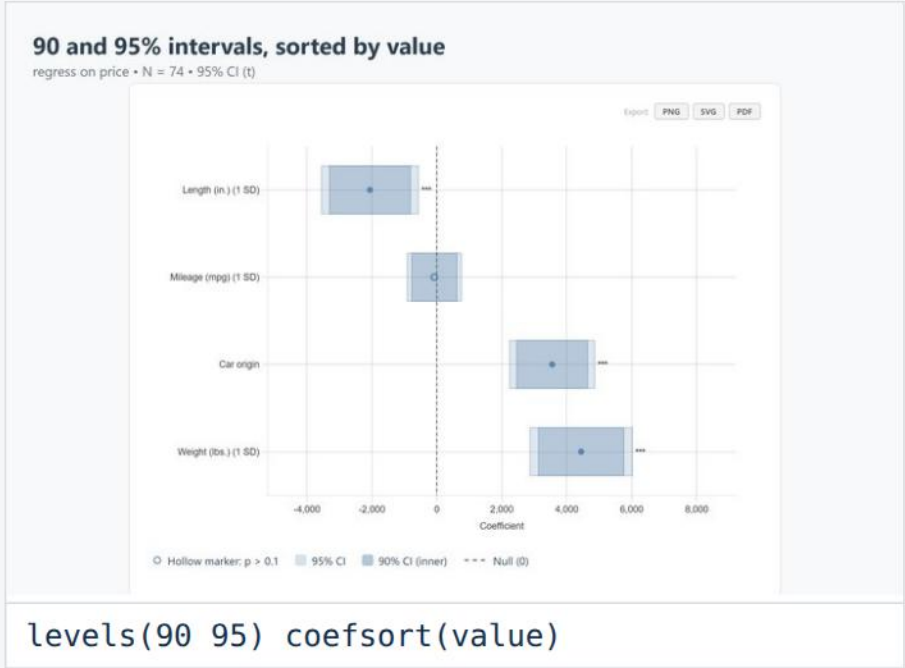
```
. regress price z_mpg z_weight z_length foreign
. estimates store full
. regress price z_mpg z_weight
. estimates store reduced
. sparkta, type(coefplot) estnames(full reduced)
```

Scroll to the table, press Copy and choose LaTeX.



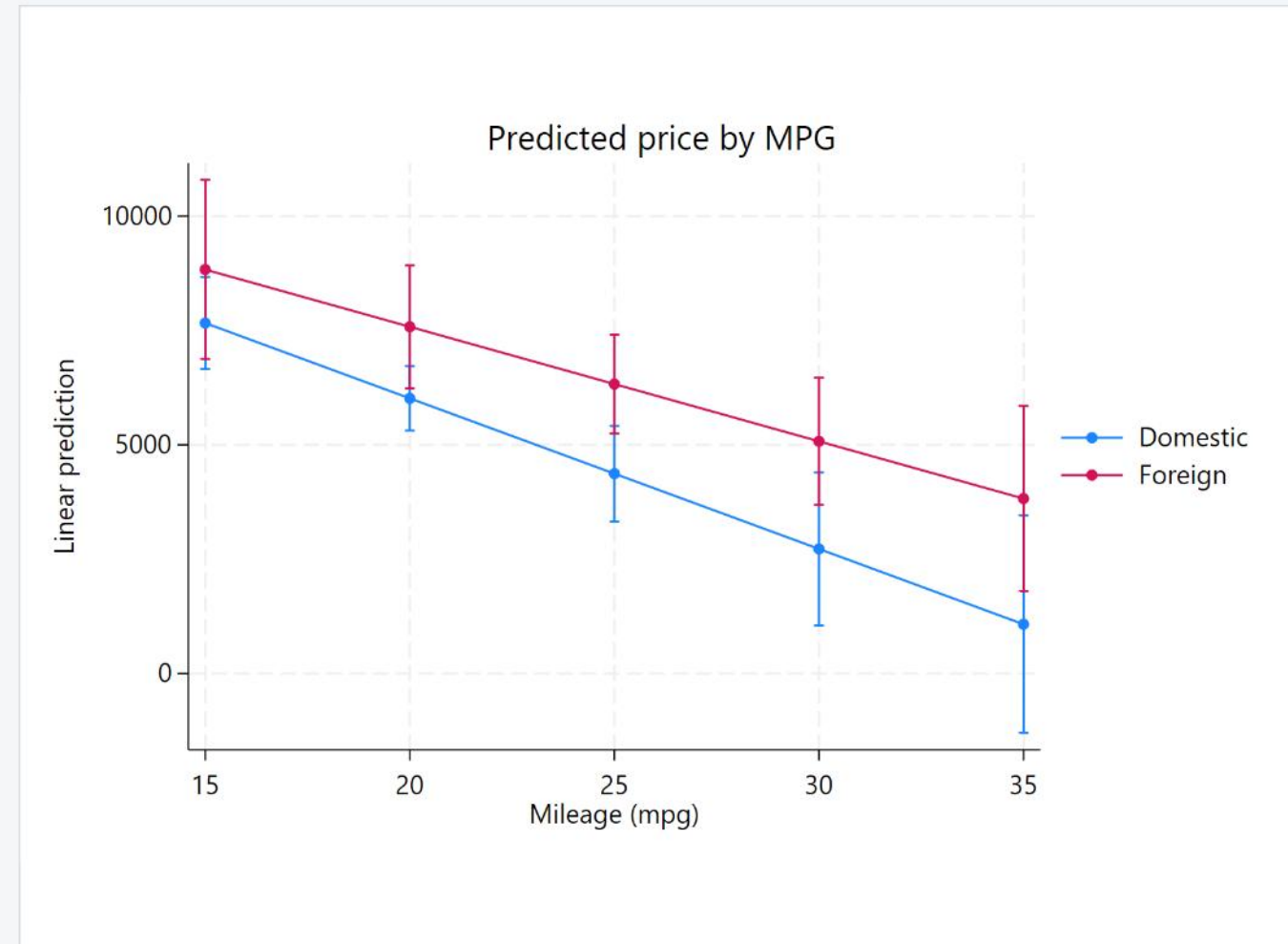
The options coefplot users expect

Click a tile to open the live page.



marginsplot, same margins call

Stata graph, a picture



```
. margins foreign, at(mpg=(15(5)35))  
. marginsplot
```

sparkta, a page you can use



```
. margins foreign, at(mpg=(15(5)35))  
. sparkta, type(marginsplot) cistyle(whisker) connected
```

Event studies

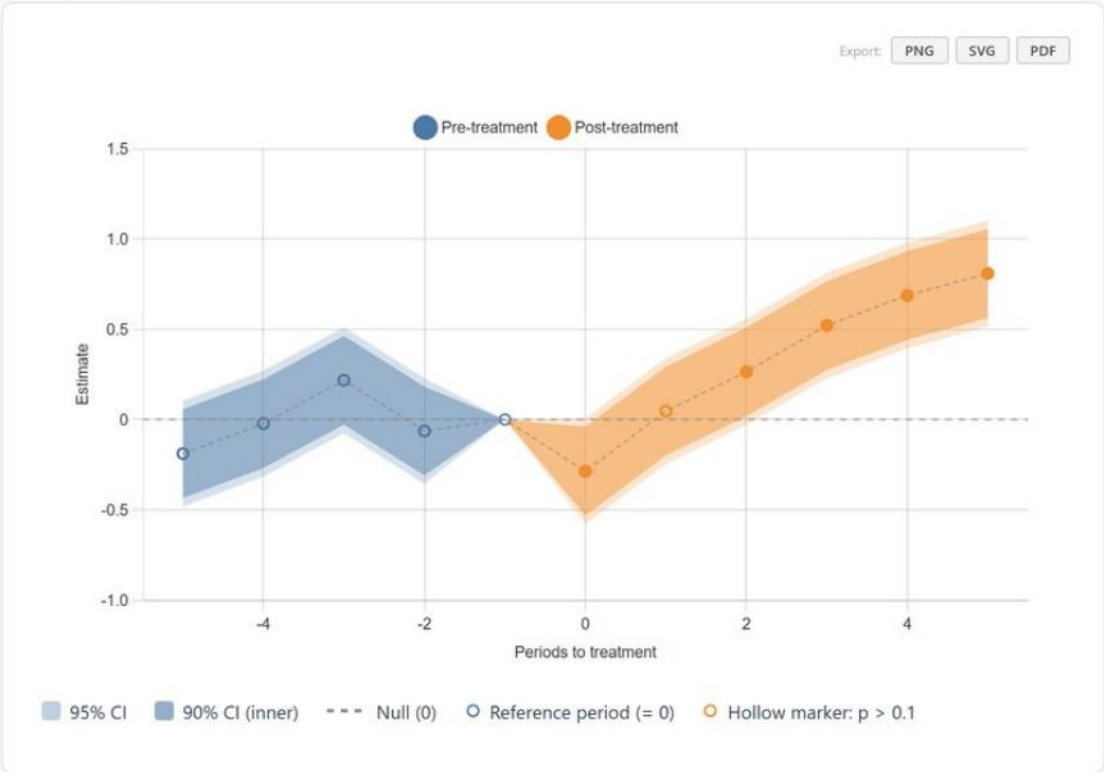
Leads and lags parsed from coefficient names, the omitted period drawn as a hollow marker at zero, pre and post in two colours, nested 90 and 95% bands.

```
. regress y lead5 lead4 lead3 lead2 lag0-lag5 i.t i.id  
. sparkta, type(eventstudy) show(lead* lag*) ///  
  cistyle(area) connected levels(90 95)
```

Hover t-2 and t+3: coefficient, CI and p for that period.

Event study: leads and lags, 90 and 95% intervals

regress on y • N = 4,000 • 95% CI (t)



Export: [Copy](#) [Download](#) View: [Publication](#) [Detailed](#)

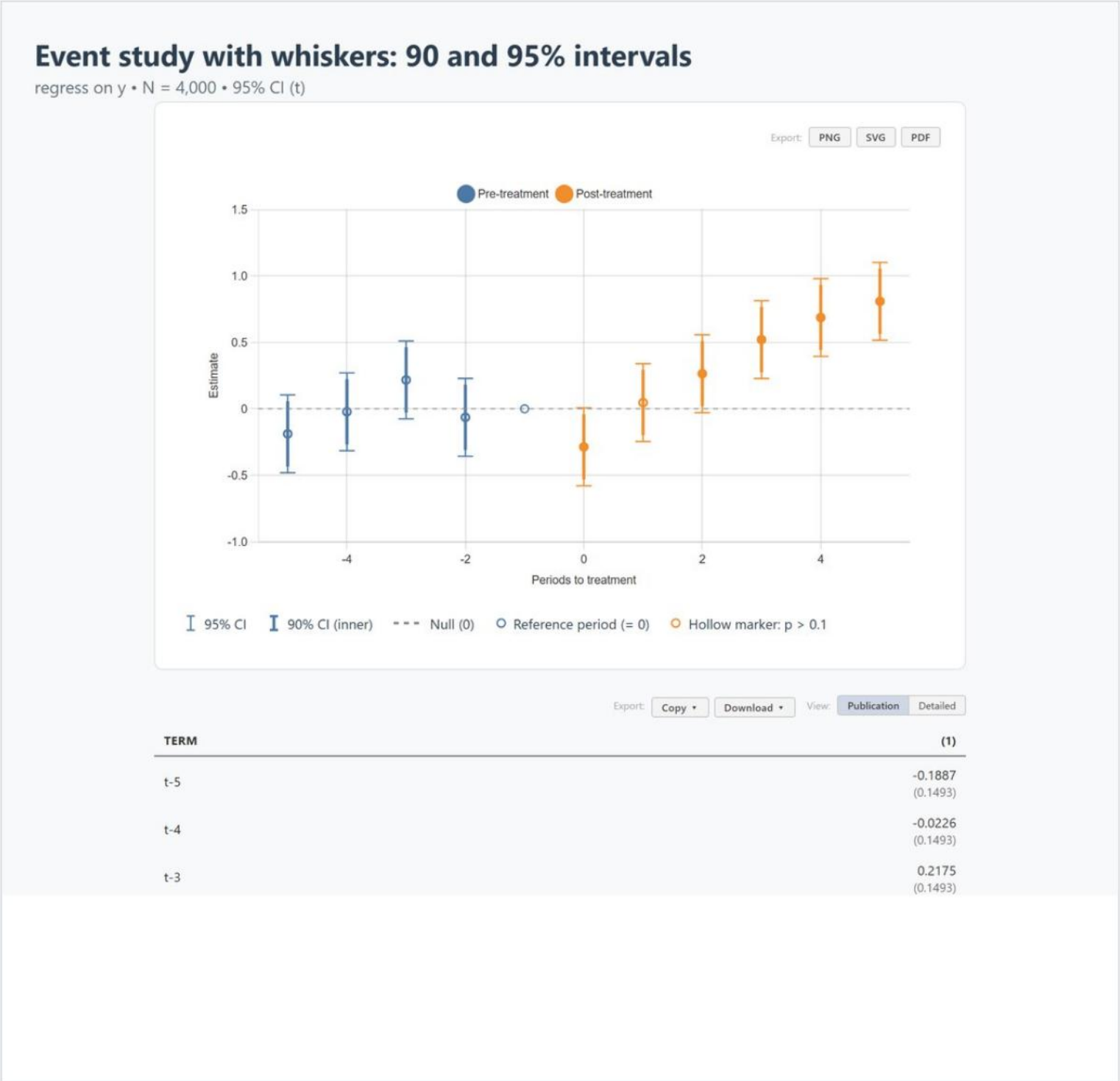
TERM	(1)
t-5	-0.1887 (0.1493)
t-4	-0.0226 (0.1493)
t-3	0.2175 (0.1493)

The same event study, with whiskers

cistyle() takes whisker, band, bar or area, so the chart can look like Stata's rcap or like a ribbon. Nothing else changes.

```
. sparkta, type(eventstudy) show(lead* lag*) ///  
  cistyle(whisker) levels(90 95)
```

Hover t+3: the same numbers as on the band version.



Draw from the model, a matrix, or a saved dataset

e()

The last estimation, or models kept with estimates store.

```
regress price mpg weight
estimates store m1
regress price mpg weight foreign
estimates store m2
```

```
sparkta, type(coefplot) ///
    estnames(m1 m2) estlabels(Base~Full)
```

regress, logit, probit, reghdfe, ivreg2 ... anything that posts e(b) and e(V). No option = the last model.

matrix()

Any results matrix: r(table), an estimator's output, or one you build.

```
csdid y, ivar(id) time(t) gvar(g)
estat event
sparkta, type(eventstudy) matrix(r(table))
```

```
* your own matrix, rows named b se
sparkta, type(coefplot) matrix(M) df(30)
```

Rows found by name (b, se, ll, ul, pvalue) as in coefplot; or say where: ci(ll ul), se(), df(), pvalue(). matrix(A B) = two models.

results()

```
results(file | frame:name [, name() b()
se() ci() pvalue() time()])
```

```
* a saved .dta file
lwdid ..., save(ev)
sparkta, type(eventstudy) results(ev)
```

```
* a frame: create it, fill it, name it
frame create est
frame est: use regsave_out, clear
sparkta, type(coefplot) results(frame:est)
```

```
* columns sparkta cannot guess: map them
sparkta, type(coefplot) ///
    results(frame:est, name(regressor) ///
        b(point) ci(lo hi))
```

parmest, regsave (var coef stderr), lwdid and margins, saving() layouts are recognised. Your data and frames are left as they were.

The model does not have to be in memory: a saved results file redraws the chart years later.

Part 5 of 5

Output and scale

Back to paper when you need it, and big data when you have it.

One option back to the journal

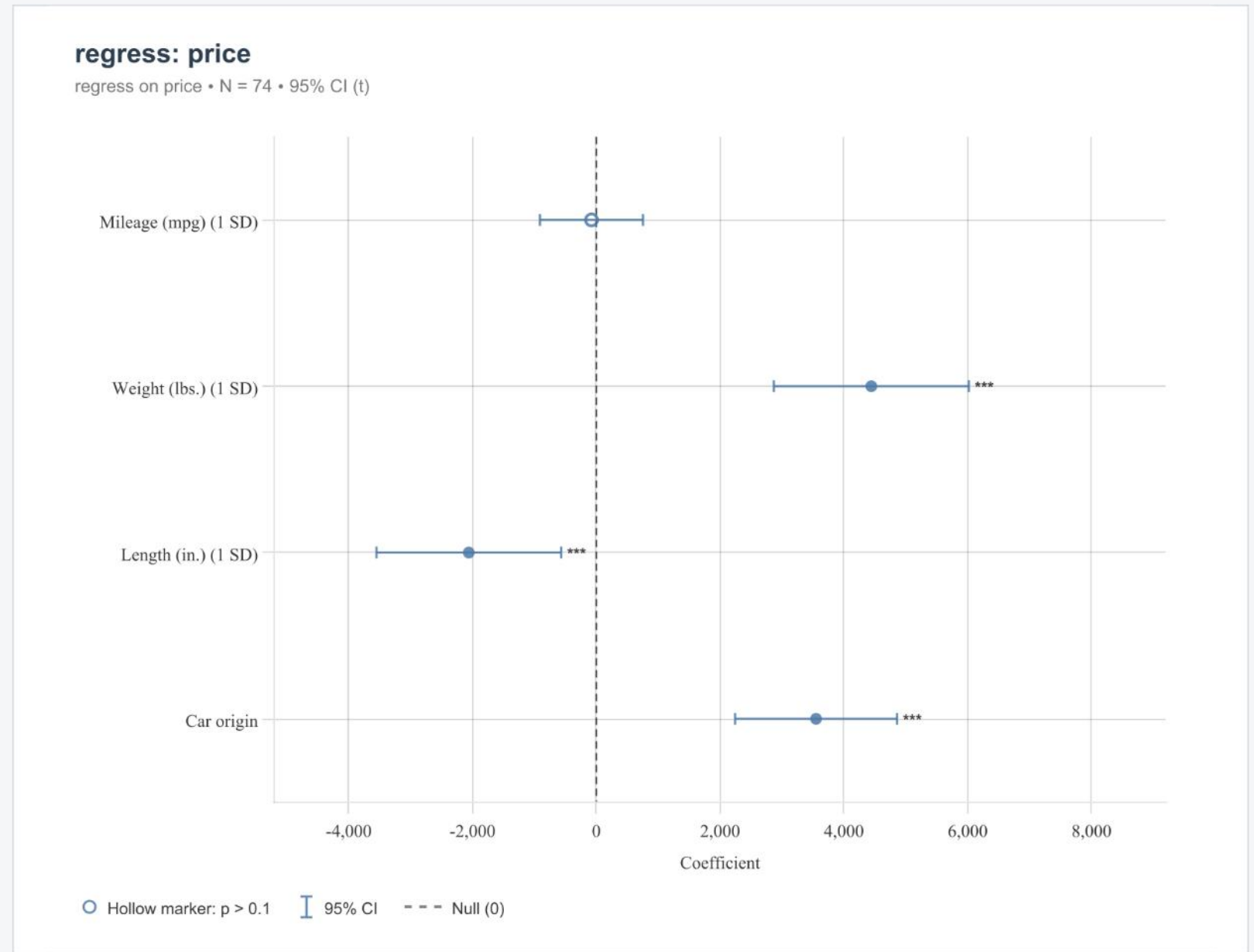
```
. sparkta, type(coefplot) export(coef.html)
///  
saveas(coef.png coef.pdf coef.svg)
```

```
. sparkta, type(coefplot) export(coef.html)
///  
saveas(coef_table.pdf, table)
```

PNG at any scale; PDF and SVG stay vector

Open the results: [PDF](#), [SVG](#), [table PDF](#)

Headless Edge or Chrome; the browser closes itself when done



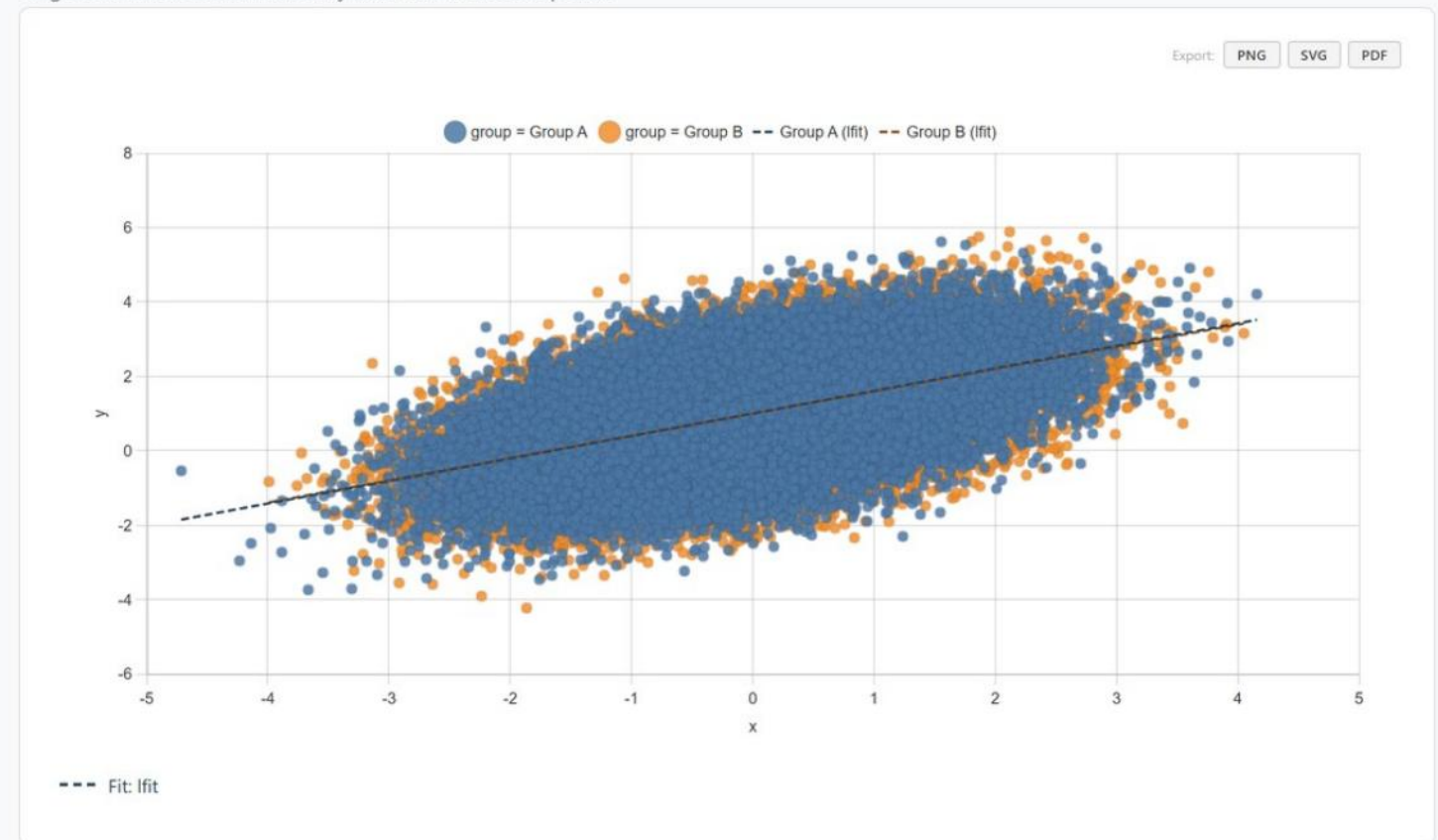
100,000 points, still interactive

Above 20,000 points sparkta switches to large-data mode by itself: the cloud is drawn once, hover and export keep working. This page, 4.8 MB, was built in about 6 seconds.

```
. set obs 100000  
...  
. sparkta y x, type(scatter) over(group) fit(lfit)
```

100,000 points, still interactive

Large-data mode switches on by itself above 20,000 points



Hover the dense middle: the tooltip still finds single points.

The options, grouped as in help sparkta (1 of 2)

Charts from data. Stata words such as `keep()`, `drop()`, `level()`, `msymbol()`, `lwidth()` are accepted as aliases.

Essential

`type()` `over()` `by()` `filters()` `sliders()`
`stat()` `title()` `export()` `saveas()`

Panels and grouping

`layout()` `yfree` `sortgroups()` `nomissing`
`novaluelabels` `noallfilter` `nostats`
`collapsestats`

Axes

`xtitle()` `ytitle()` `xrange()` `yrange()`
`ystart(zero)` `xtype()` `ytype()` `xreverse`
`xticks()` `xlabels()` `xtickangle()` `ygrace()`
`y2()` `y2title()`

Titles and text

`subtitle()` `note()` `caption()` `notimestamp`
`datalabels` `pielabels` `titlesize()`
`titlecolor()` `xlabsize()` `ylabsize()`

Legend

`legend(top|bottom|left|right|none)` `nolegend`
`leglabels()` `legtitle()` `legsize()` `legcolor()`
`legbgcolor()`

Colours and theme

`theme()` `colors()` `mcolor()` `palette()`
`bgcolor()` `plotcolor()` `opacity()` `gradient`
`gradcolors()`

Bars

`horizontal` `stacked` `barwidth()`
`bargroupwidth()` `borderradius()`

Lines and points

`fill` `areaopacity()` `smooth()` `stepped()`
`spanmissing` `lwidth()` `lpattern()` `msize()`
`msymbol()` `nopoints`

Scatter and bubble

`fit(lfit|qfit|lowess|exp|log|power|ma)`
`fitci` `mlabel()` `mlabpos()` `mlabvposition()`

The options, grouped as in help sparkta (2 of 2)

Distributions, annotations, page settings, and everything after an estimation command. help sparkta lists all 215 with clickable examples.

Distributions <code>bins()</code> <code>histtype()</code> <code>whiskerfence()</code> <code>bandwidth()</code> <code>mediancolor()</code> <code>meancolor()</code> <code>cibandopacity()</code>	Pie and donut <code>cutout()</code> <code>rotation()</code> <code>circumference()</code> <code>sliceborder()</code> <code>hoveroffset()</code>	Reference lines and annotations <code>yline()</code> <code>xline()</code> <code>ylinelabel()</code> <code>yband()</code> <code>xband()</code> <code>apoint()</code> <code>alabelpos()</code> <code>alabeltext()</code> <code>alabelgap()</code> <code>aellipse()</code>
Tooltip and animation <code>tooltipformat()</code> <code>tooltipmode()</code> <code>tooltipbg()</code> <code>animate()</code> <code>animduration()</code> <code>easing()</code> <code>aspect()</code> <code>padding()</code>	Page, export and browser <code>online</code> <code>offline</code> <code>notoolbar</code> <code>browser()</code> <code>findbrowser</code> <code>closebrowser</code>	Post-estimation input <code>estnames()</code> <code>matrix()</code> <code>results()</code> <code>ci()</code> <code>se()</code> <code>df()</code> <code>pvalue()</code>
Coefficient selection and labels <code>keep()</code> <code>drop()</code> <code>nocons</code> <code>nobase</code> <code>order()</code> <code>coefsort()</code> <code>coeflabels()</code> <code>headings()</code> <code>estlabels()</code>	Post-estimation look <code>eform</code> <code>rescale()</code> <code>level()</code> <code>levels()</code> <code>cistyle()</code> <code>coefstyle()</code> <code>connected</code> <code>vertical</code> <code>refval()</code> <code>cicolors()</code> <code>refperiod()</code>	Publication table <code>stars()</code> <code>nostars</code> <code>tstat</code> <code>interval</code> <code>nofooter</code> <code>notable</code> <code>indicators()</code> <code>addstats()</code>

3.6 is a public beta: more is coming

Version	What
3.6.1	the fixes found while preparing this talk; feedback from today
3.7	overlay charts (bar + line, histogram + density), zoom and pan, more chart types, faster very large scatters
3.8	sparkta_dash: several charts on one page with shared filters
3.9 and 4.0	scrollytelling pages, heatmaps, forest plots; frames integration, user themes

Options may still change before 4.0. Issues and pull requests are welcome on GitHub (MIT licence).

Try it today

```
net install sparkta, from("https://raw.githubusercontent.com/fahad-mirza/sparkta_stata/main/ado/") replace
```

```
ssc install sparkta once the SSC archive lists it
```

Stata 17 or later; nothing else to install. Tested on Windows; Mac and Linux run the same jar (feedback welcome). `help sparkta` has every option with clickable examples.

Thank you

Code, help, examples and the gallery on GitHub

github.com/fahad-mirza/sparkta_stata

medium.com/@fahad-mirza

fm.109@hotmail.com



Scan for the GitHub repository